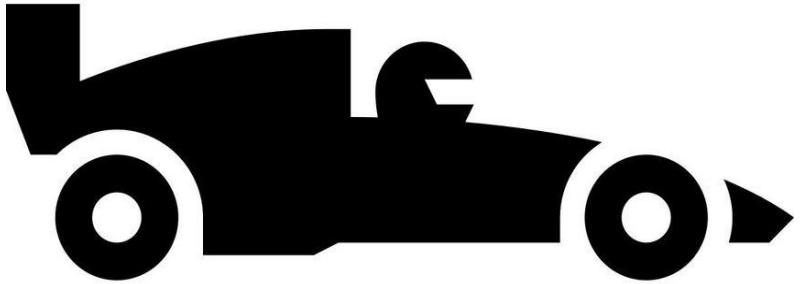
FORMULA 1 API

A RESTful Web Service for Formula 1 Data

PROJECT OVERVIEW



Dataset: Ergast Motor Racing – Kaggle

- Data Ranges from 1950-2024
- Includes 900+ Drivers, 300+ Teams, and 1100+ Races

24 RESTful endpoints for CRUD and Analytical Queries

- Championship Standings, Leaderboards, Career Summaries

API Key authentication on all write operations

Live Deployment with Swagger UI

TECHNOLOGY STACK

- **FastAPI**

Built-in Swagger UI at /docs (Primary manual testing tool)

Pydantic validation removes manual input checking

Widely adopted, well-documented and maintained

- **SQLite**

Serverless single-file, easy to deploy alongside app

API is read-heavy (single-writer limit not a major concern)

No separate Server Process (via PostgreSQL)

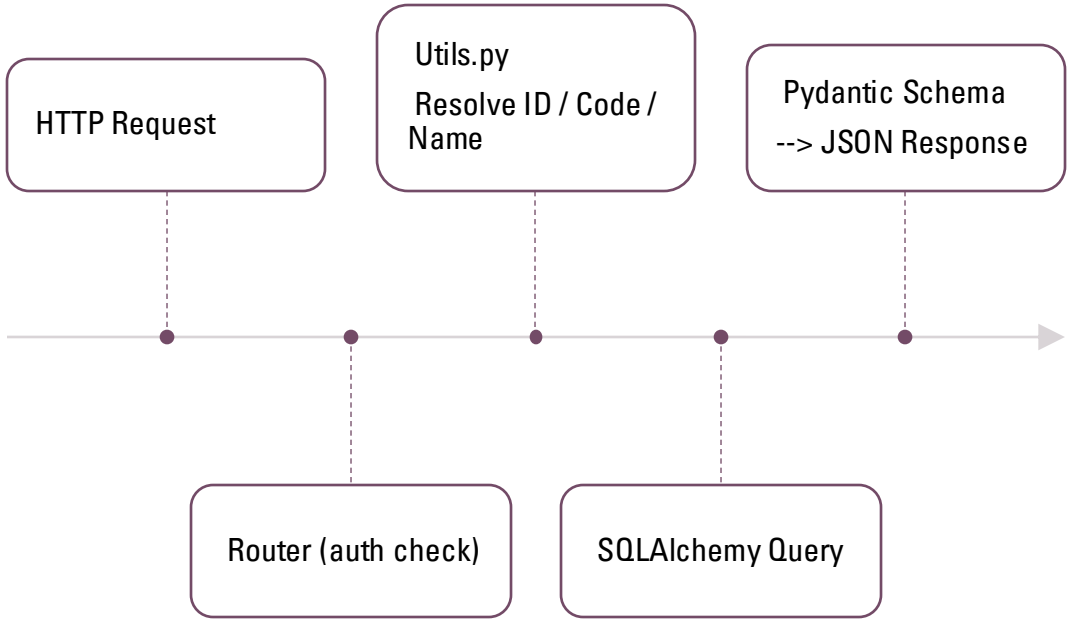
- **SQLAlchemy ORM**

Manages relationships between tables in Python

Raw SQL across 6 related tables becomes unsustainable

Future scope: PostgreSQL migration requires minimal code change

ARCHITECTURE AND DESIGN DECISIONS



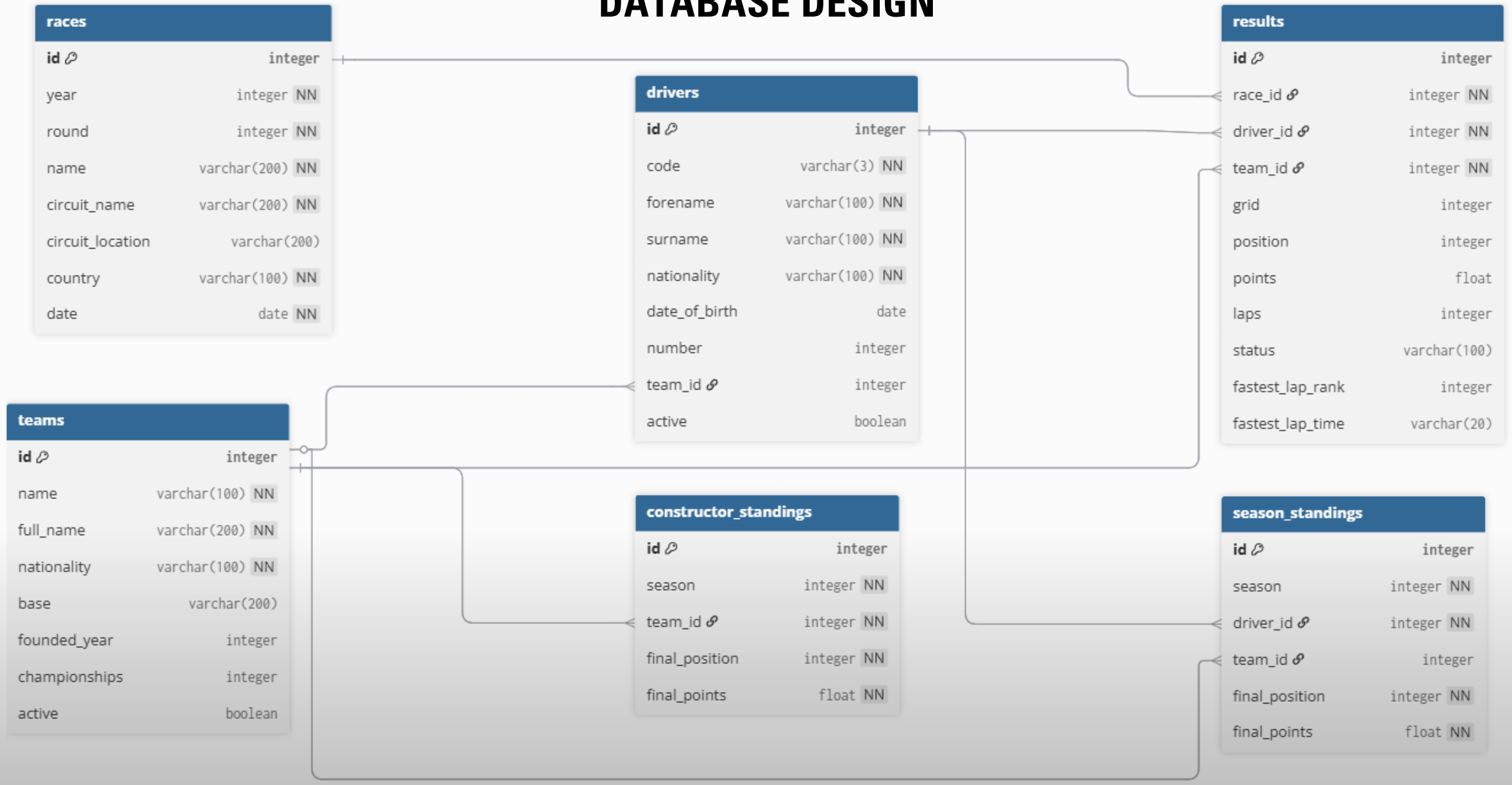
No Service Layer:
- CRUD on static data has no business logic
- Extra layers add no value

Utils.py:
- Single resolve_driver() / resolve_team() used across all routers

Auth.py:
- Secrets.compare_digest() for constant-time comparison
- Helps against timing attack resistance

Pydantic Schemas:
- Centralise validation and response shaping automatically

DATABASE DESIGN



dbdiagram.io

TESTING

Test Setup:

- Pytest + FastAPI TestClient
- No live server required
  - Fast and self-contained
- Fresh in-memory SQLite database per session
  - Tests do not touch the production database
- Conftest.py: Shared DB setup and test client

Coverage

41 Tests across 5 files

GET list endpoints with query filters

GET by ID, driver code, and name

POST creation with response validation

Duplicate rejection (409)

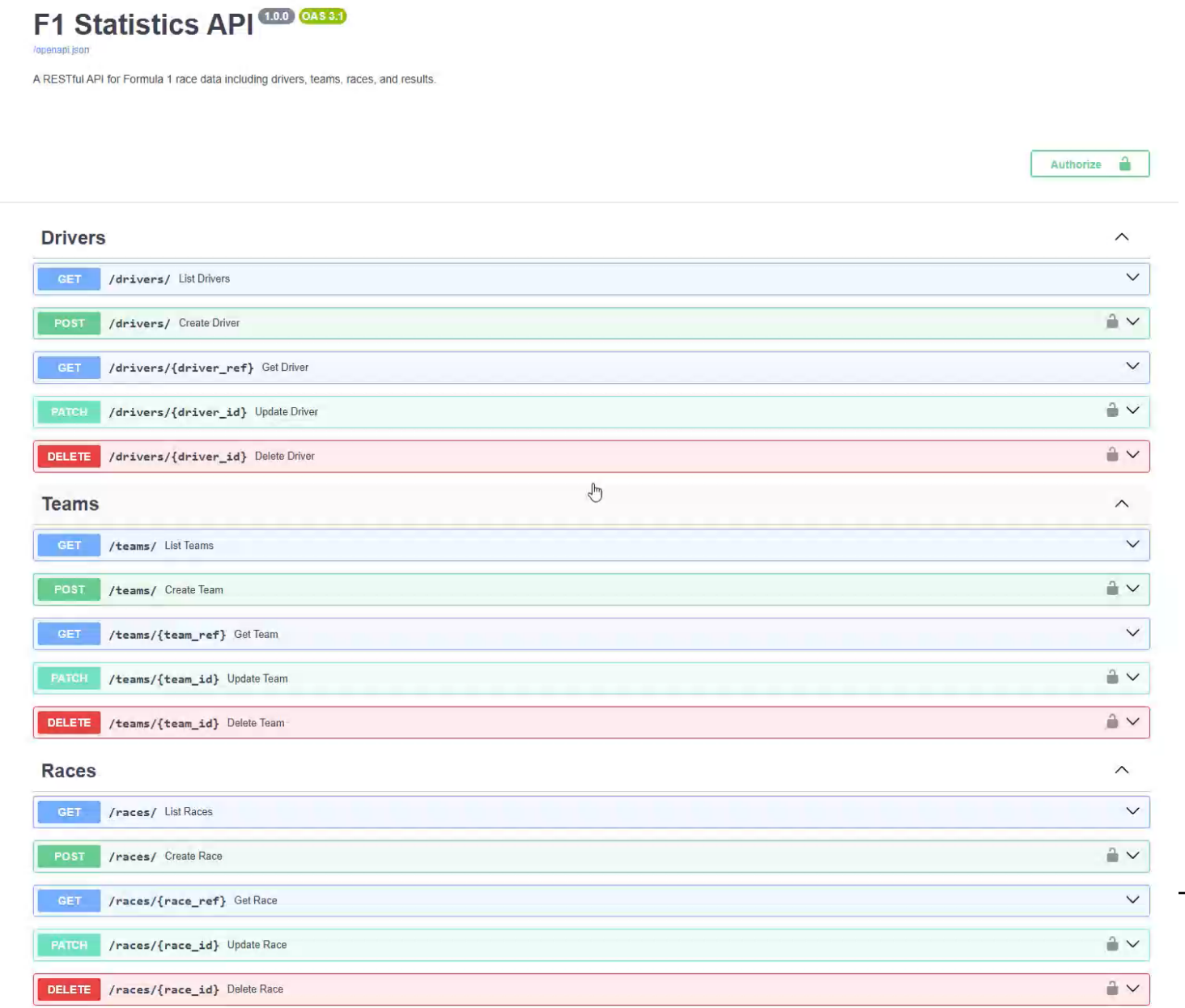
Foreign key violations (404/422)

PATCH partial updates

DELETE operations

Analysis: standings, leaderboard, career

API DOCUMENTATION



Swagger UI

- Auto generated by FastAPI

5 Resource Groups – **24 Endpoints**

- Drivers, Teams, Races, Results
- Full CRUD (GET, POST, PATCH, DELETE)
- Analysis (Standings, All-time leaderboard, career summaries)
- Flexible Identifiers (/drivers/HAM, /drivers/Hamilton, /drivers/1)

Authentication

- GET endpoints open, no key required
- For POST / PATCH / DELETE: API Key Required

VERSION CONTROL & DEPLOYMENT

Commit History

24 Overall Commits

Init	Project Structure, Libraries
Models	ORM Models: Teams, Drivers, Races, Results
Routers	CRUD Routers: Teams, Drivers, Races, Results
Analysis	Analysis Router: Standings, Leaderboard, Career
Seed	Seed.py - Ergast Dataset Import
Tests	41 Pytest tests with in-memory database
Auth	API Key Authentication for write operations
Utils	Multi-Identifier Resolution via utils.py
Deploy	PythonAnywhere
Docs	README + API Documentation

Deployment

PythonAnywhere – Free Python Hosting

Issue 1: SQLite path broke in production
Fix: Absolute path derived from __file__

Issue 2: WSGI wrapper caused uWSGI hang
Fix: Deploy via pa website create with Uvicorn Directly

Workflow after deployment:
-> Develop Locally
-> Git Push
-> Git Pull on PythonAnywhere terminal
-> pa website reload

REFLECTIONS & LESSONS LEARNED

Domain Knowledge Importance

Assumed Unique Driver Numbers
- Not the case pre-2014

Aggregated Point totals ≠ Official Standings
- Dropped scores, countbacks

Led to separate SeasonStanding/ConstructorStanding models

Results model Complexity

Position Nullable
- Retired drivers have no finishing position

Points as float
- Half-points in shortened races

Duplicate Check in router
- Same driver cannot appear twice in a race

AI-Assisted Development

Heavily used to explore alternatives

Consistently used to identify weaknesses and correct flaws

Troubleshooting throughout (e.g. Deployment)

DELIVERABLES

GitHub Repository

github.com/kyrikonis/F1_API

Public repo with full commit history

README with setup instructions

API documentation PDF included

Live Deployment

kyrik2005.pythonanywhere.com

Swagger UI available at /docs

All endpoints accessible

API key required for write operations

Technical Report

Submitted via Minerva

Tech Stack Justification

Design & Architecture

Reflections

Limitations

GenAI Declaration with full chat logs

THANK YOU